

Arquitectura Web en 3 Capes



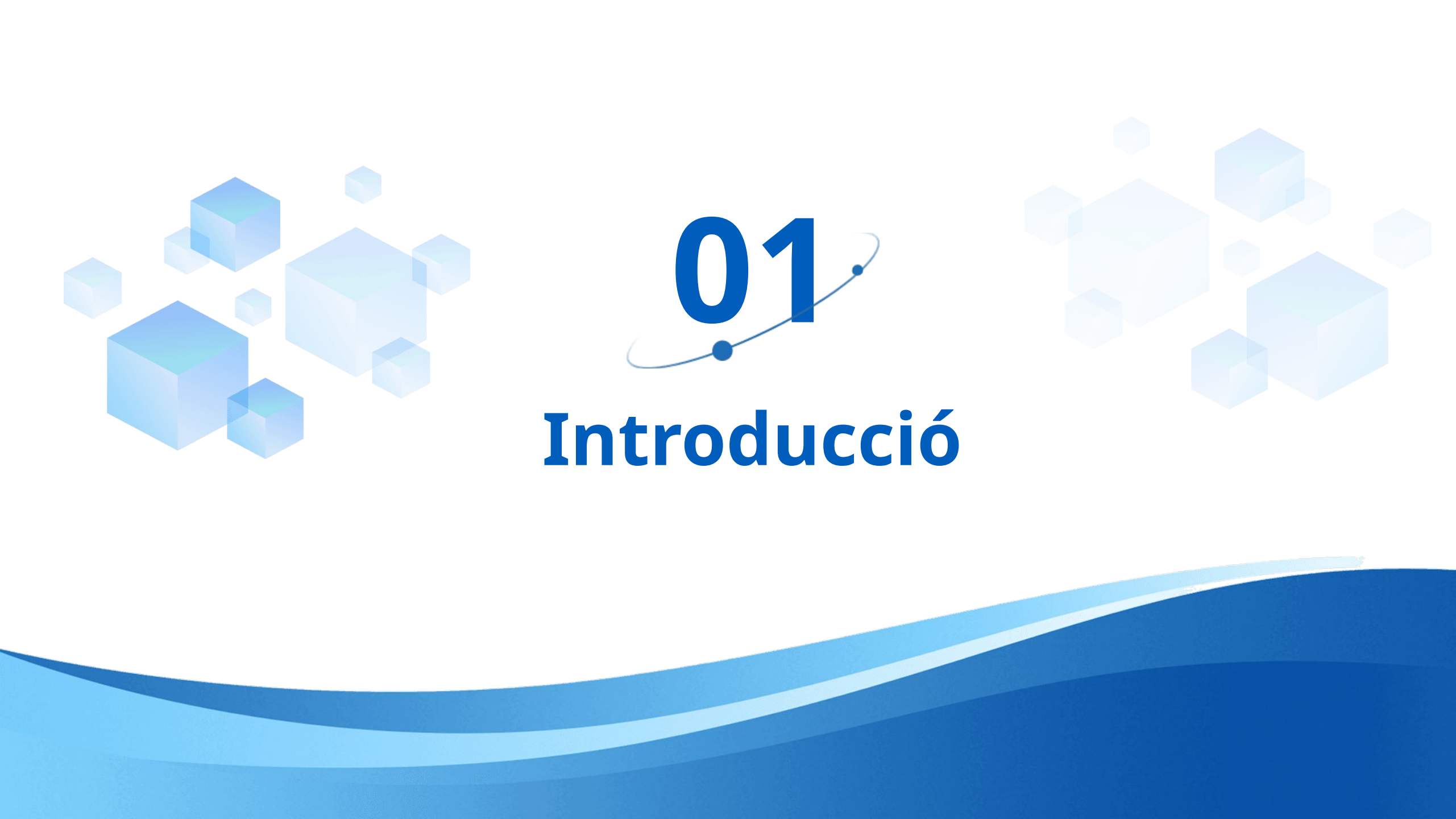
RA1



Continguts

- ▶▶ 01 Introducció
- ▶▶ 02 Capa de Presentació
- ▶▶ 03 Capa de Lògica
- ▶▶ 04 Capa de Dades
- ▶▶ 05 Integració i Escalat
- ▶▶ 06 Casos i Bones Pràctiques





01

Introducció

Què és l'arquitectura de 3 capes

Definició

L'arquitectura de tres capes divideix una aplicació web en presentació, lògica i dades. Cada capa té funcions específiques que faciliten el desenvolupament i el manteniment de l'aplicació.

Separació de responsabilitats

Aquest model separa les responsabilitats entre les diferents capes, permetent que cada una es dediqui a una tasca específica. Això simplifica el codi i el fa més fàcil de gestionar.

Beneficis

La separació de capes permet actualitzacions independents sense afectar les altres capes, millora l'escalabilitat i facilita el treball en equips multidisciplinaris.



Avantatges del model per capes

Reducció de complexitat

Separar la interfície, el negoci i l'emmagatzematge redueix la complexitat del codi, facilitant el desenvolupament i el manteniment de l'aplicació.

Seguretat i reutilització

Aquest model augmenta la seguretat, permet equips especialitzats i simplifica les proves. També facilita la reutilització de components entre diferents parts de l'aplicació.





02

Capa de Presentació



Funció i responsabilitats del frontend

Interfície d'usuari

La capa de presentació és el punt de contacte amb l'usuari final. S'encarrega de renderitzar les interfícies gràfiques i proporcionar una experiència fluida.

Validació bàsica

Realitza validacions bàsiques de les entrades de l'usuari abans de enviar les dades al backend, millorant l'experiència d'ús i reduint el trànsit de dades incorrectes.



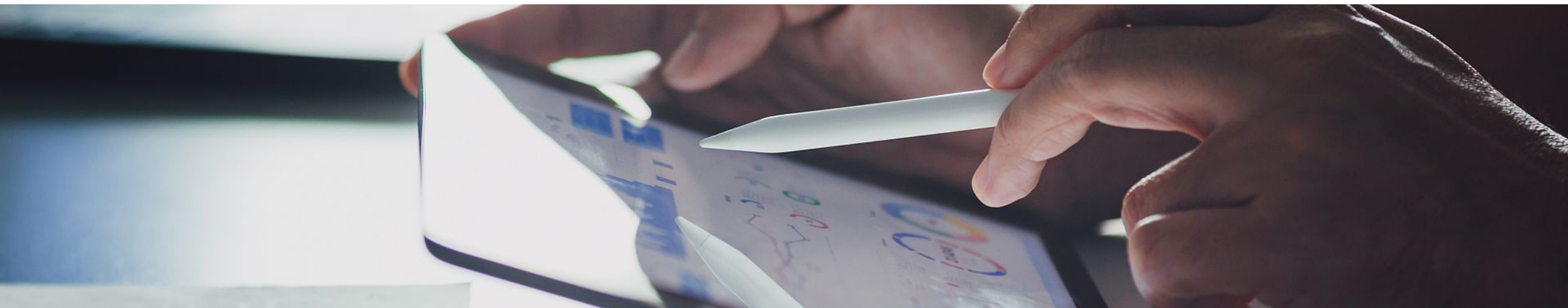
Gestió d'estat

Gestiona l'estat de les pantalles i les interaccions de l'usuari, garantint que l'aplicació respon de manera adequada a les seves accions.

Comunicació amb backend

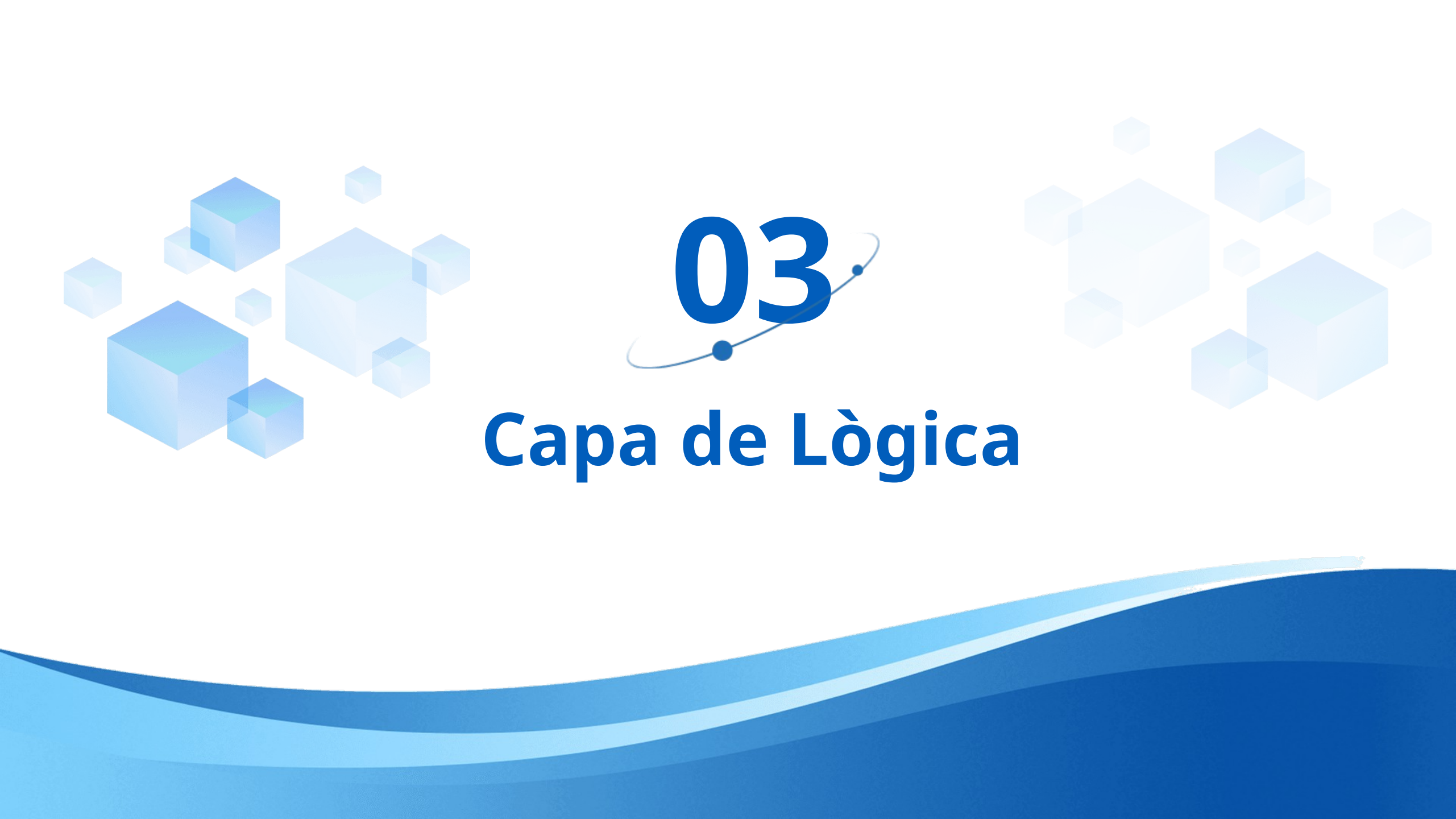
S'encarrega de comunicar-se amb la capa de lògica mitjançant crides HTTP o WebSocket, sense emmagatzemar dades sensibles en el client.

Tecnologies habituals al client



Tecnologies bàsiques i avançades

HTML, CSS i JavaScript són les tecnologies bàsiques, mentre que frameworks com React, Vue i Angular afegeixen reactivitat i components reutilitzables. TypeScript millora la mantenibilitat del codi.



03

Capa de Lògica



El rol del backend a l'arquitectura

Processament de peticions

La capa de lògica rep peticions del frontend, executa les regles de negoci i autoritza operacions, assegurant que les accions de l'usuari són vàlides i segures.

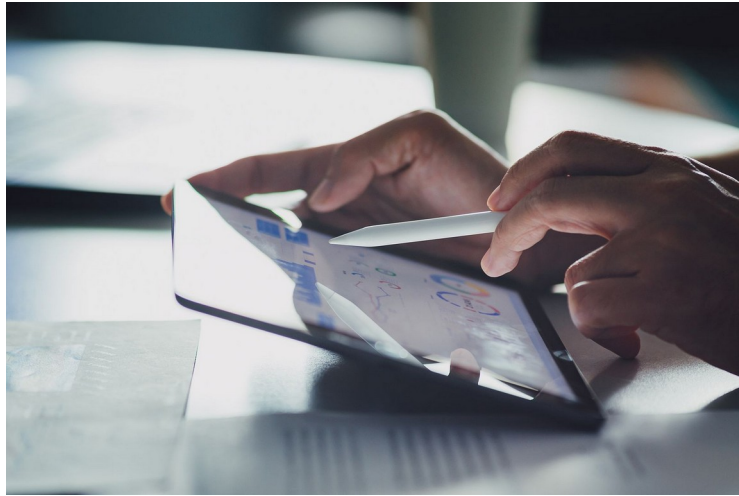
Intermediari entre capes

Actua com a intermediari entre la capa de presentació i la capa de dades, garantint la consistència de les dades i encapsulant la complexitat del domini en serveis reutilitzables.

Disseny d'APIs REST i GraphQL

APIs REST

Les APIs REST utilitzen verbs HTTP i codis d'estat estàndard per organitzar recursos de manera clara i previsible, facilitant la comunicació amb el frontend.



GraphQL

GraphQL ofereix consultes declaratives que permeten al frontend sol·licitar exactament les dades necessàries, reduint l'over-fetching i millorant l'eficiència.



Documentació i autenticació

Tant les APIs REST com les GraphQL requereixen documentació clara i mètodes d'autenticació per garantir que el frontend pugui accedir de manera segura.

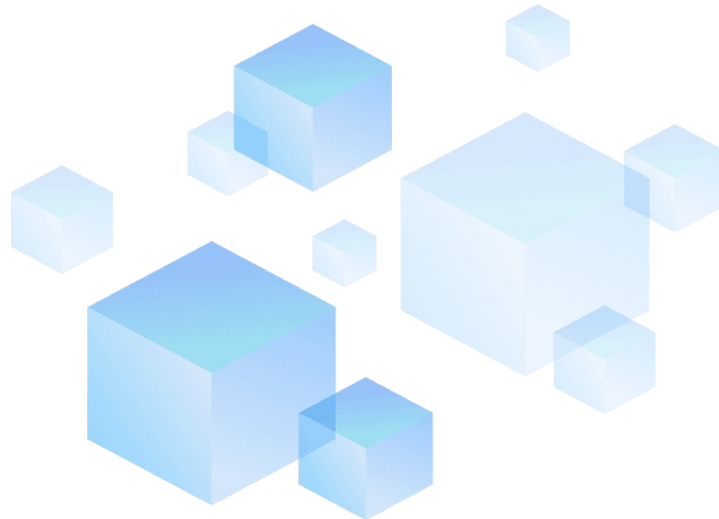


Seguretat i validació centralitzada

Centralització de seguretat



Totes les regles d'autorització i validació es concentren al backend, evitant duplicacions i assegurant que les dades són processades de manera segura i consistent.



04

Capa de Dades



Persistència i modelatge de la informació



Emmagatzematge de dades

La capa de dades gestiona l'emmagatzematge de les dades mitjançant bases de dades relacionals, NoSQL o una combinació de les dues, segons les necessitats de l'aplicació.

Modelatge de dades

Defineix esquemes, índexs i relacions que reflecteixen el domini de l'aplicació, assegurant la integritat referencial i facilitant l'evolució del sistema.



ORM, consultes i transaccions



01

ORM

Mapejadors d'objectes relacional (ORM) com Hibernate o Sequelize abstraient SQL i acceleren el desenvolupament, però s'han d'utilitzar amb cura per evitar problemes com les consultes N+1.

02

Consultes eficients

Les consultes ben dissenyades són essencials per obtenir les dades necessàries de manera eficient, evitant càrregues excessives de dades i millorant el rendiment.

03

Transaccions

Les transaccions asseguren que les operacions són executades de manera atòmica, preservant la consistència de les dades en cas d'errors o interrupcions.



05

Integració i Escalat



Comunicació entre capes



Protocols de comunicació

La comunicació entre capes es realitza mitjançant protocols com HTTP/JSON o gRPC, mentre que les cues de missatges permeten operacions asíncrones i milloren l'escalabilitat.

Escalabilitat i desplegament al núvol



Elasticitat

Cada capa es pot replicar en contenidors orquestrats per Kubernetes, permetent l'escalat horitzontal segons la demanda i millorant la disponibilitat.

Optimització de recursos

El balanceig de càrrega i les lectures de només lectura en bases de dades optimitzen l'ús de recursos, assegurant un rendiment consistent sota càrregues altes.



06

Casos d'ús i Bones Pràctiques



Exemple pràctic i lliçons clau



Exemple d'e-commerce

Un exemple típic és un e-commerce que utilitza React al frontend, microserveis Spring al backend i PostgreSQL a les dades, demostrant la separació clara de responsabilitats.

Escalat selectiu

Separar funcionalitats com l'autenticació, la cistella i els pagaments en serveis independents facilita l'escalat selectiu segons les necessitats de l'aplicació.

Bones pràctiques

La clau és mantenir interfícies estables i realitzar tests automatitzats per evitar regressions, assegurant una arquitectura robusta i escalable.



Gràcies

RA1

